Analysis of Distributed Denial of Service Attacks: An Internal Perspective

Jeremy Middleman*, Ben Ogden†, Matthew Rackley‡, Loc Su§ and Christopher Tye¶
Dept. of Computer Science, University of New Mexico
Albuquerque, NM
*jiddleman@unm.edu, †btoraptor@unm.edu, ‡alimar@cs.unm.edu, §locsu@unm.edu, ¶ctye@unm.edu

***Abstract*—Motivation: In this paper we intend to investigate distributed denial of service attacks. Also known as DDoS attacks, these attacks are commonplace every day and threaten to affect both small and large scale services across all IoT (Internet of Things) devices. By having a twofold implementation of research; theory and empirical. We aim to understand the nature of DDoS attacks and shed some light on their structure and behavior.**

Methods: For the theory section, we aim to study both the nature of attackers and defenders. To do this looking into real world analysis of historical and modern attacks and defensive methods employed by known companies such as Google, Amazon and Github are used. To support our theory section, implementation of attacker and defender dockers will be used to get an estimation of how DDoS attacks behave and how the victim server responds, as well as application against a live webpage behind a Cloudflare proxy.

Results: We found that increasing the concurrency of requests up to 10,000 substantially increased the number of failed requests, decreased the tranfer rate, and increased the total time to complete the request. DDoS attacks on the Docker container proxies increased the mean and variance for the connection time. On the website, increasing the number of concurrent requests to 10,000 with 100,000 total requests revealed a threshold at which point the total time to process all requests spiked.

Conclusions: The research contained within this report will serve as a general guide to introductory DDoS knowledge and techniques, providing a entry point to defensive studies by thoroughly understanding attack threats.

I. Problem Statement

As our lives and data increase in integration, so do the vulnerabilities we face in our applications and systems. Vitally, the proliferation of Distributed Denial of Service (DDoS) attacks presents precisely such a challenge. By taking advantage of services in their intended form, these attacks present themselves as simple to perform, but destructive to resources all the same. In an attack vector that will remain as open as its victim service does, offensive techniques are developing at unprecedented rates. Diversity in attacks is as prevalent as ever as seen in Figure 1 [16].

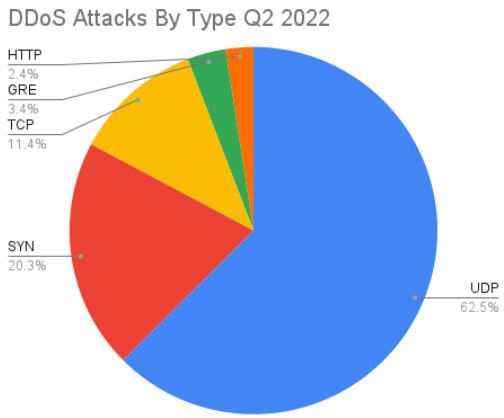

Fig. 1: Distribution of DDoS attack types across the globe

In this study, we aim to assume the position of a malicious party and dive into the intricacies and deployment of various DDoS attack techniques, taking care to ethically measure their impact on servers. This report employs a multifaceted approach to understanding the technical aspects of attacks as well as what can be done to defend against such attacks by these malicious agents. Our investigation is structured in two phases, theory and application.

In the theory phase we plan on analyzing the basic descriptions, taxonomy and methods of DDoS attacks so that we have a basic grasp of the definitions and nature of them. This will include both attacks and defense methods by using peer reviewed research to bootstrap our knowledge. Furthermore, we intend to explore historical case studies to provide context and perspective on the evolution of DDoS attacks.

While large-scale attacks against prominent organizations garner significant attention, we recognize the importance of examining lesser-known incidents targeting smaller entities. By analyzing these case studies, we seek to expand the nuances of offensive capabilities and identify potential strategies for enhancing defensive measures.

In the application phase we plan on implementing a virtual server environment to simulate real-world conditions and attempt multiple DDoS attack vectors as researched from our theory team. Collaborating with another group, Franklin and Althea, we will conduct these experiments on their website infrastructure. Their website can found at the link https://agora-app.com/.

This collaboration not only provides a practical testing ground but also offers an ethical opportunity to assess the effectiveness of DDoS mitigation measures, particularly in the context of Cloudflare proxy servers. By adopting the perspective of potential attackers, we aim to uncover the inherent challenges and vulnerabilities present in modern web architectures.

II. Motivation

With research motivated by global tensions, it's seen that the past few years in particular have been dominated by all types of cyberattacks, including DDoS attacks. Specifically the conflicts in Ukraine and Israel have caused spikes in DDoS attacks on vital infrastructure such as Ukraine's postal service or Lithuania's Secure National Data Transfer Network [16].

Back home, it's also seen that about 45% of **all** DDoS attacks globally are waged on the United States, whereas the next highest individual country is China with 7.67% [16]. Although perhaps less glamorous than other kinds of attacks, its apparent that DDoS attacks are an instrumental weapon for malicious entities and as such motivates this reports research into attack and defense vectors.

III. Contributions

The research presented in this paper tackles a number of topics that are by no means novel alone, but expose interesting perspectives when viewed in connection with one another. Some highlights that this report provides are shown in the following.

- Introductory perspective on DDoS attacks with no prior knowledge assumption
- In depth analysis of current and historical developments in the field, particularly in defensive botnets
- Step by step discussion on a proxies affect and behavior on attack application
- Mitigative techniques derived from actual experience against a peer's webpage

IV. Hypothesis and Implications

In an effort to build a baseline for expected results and behavior, the following hypothesis will be used to guide research and script development moving forward.

Based on external knowledge and experiences as well as class instruction. We hypothesis that the DDoS attacks on real life targets is affected by a multitude of factors both internal and external to the attacker and the defender. These include but are not limited to the server vulnerability, the victim's robustness to flooding and early detection methods. On the attacker side we assume their effectiveness ranges from being able to distribute their machine attacks across a variety of machines as well as being limited on where, when and how an attack can occur. Such optimizations can be made by the attacker to get a higher success rate in attacking if they know ahead of time the infrastructure of the victim which may be compromised by other machines early on in order to gain a proper understanding of their target. We likely won't be able to test anything remotely this large however this is a common sense hypothesis.

For the defender we hypothesis that using early onset detection methods such as firewalls, machine learning, and other methods will increase detection times and help minimize losses. Through our hope in setting up a docker container we hope to validate some of these

hypotheses and hopefully learn more about the structure and taxonomy of attacks that occur.

The hypothesis above can be summed up as follows:

- Baseline protective measures against DDoS attacks are highly effective against naive attack measures
- Sites not provided by a large host are likely to be more vulnerable due to needing manual defensive implementations
- Concentrated effort and learning/training is required to wage effective DDoS attacks
- Successful attacks on larger providers either require a highly exploitable vulnerability or massive collaboration on scale

V. Methodology

A. Environment Creation

Our group used Docker containers to execute our DDoS attacks. We used them because they are lightweight and efficient, requiring minimal resources to run. Thus we can easily create multiple containers, simulating numerous attacking machines without incurring significant overhead costs. Additionally, Docker's containerization technology provides a consistent and isolated environment for running our attack scripts, ensuring reproducibility and ease of management.

Another key benefit of using Docker containers is the ability to conduct test simulations of our DDoS attack scripts before deploying them against actual targets. By setting up a victim container and one or more attacker containers, we can simulate the behavior of our attack scripts in a controlled environment. This allows us to fine-tune and validate the effectiveness of our scripts before subjecting them to live targets, minimizing the risk of unintended consequences.

1) Victim Container: The victim container consists of a single HTML file, providing basic content and serving as a target for our DDoS attack simulations. The simplicity of the victim container allows us to focus on the behavior of our attack scripts without introducing unnecessary complexity. Additionally, by hosting the victim container within the same Docker network as the attacker containers, we can accurately simulate real-world network conditions and interactions.

2) Attacker Container: Each attacker container houses a shell script responsible for initiating the DDoS attack. Our group implemented a basic DDoS script using a shell script that leverages the Apache Benchmark tool to send a specified number of requests to a target IP address. This script utilizes concurrency and repetition parameters to simulate a distributed denial of service attack. By capturing metrics such as response times and packet statistics, we can assess the impact of the attack on the target and gather data for analysis.

The Apache Benchmark script however has the issue that it will be unable to reach the target website. We believe that simply sending thousands of request messages will most likely cause a outside network, in between our group and the website, to detect a DDoS attack from the mere bulk of messages being sent and will start to get discarded, causing our DDoS attack to fail before getting to the website. To combat this, our group looked into implementing a python SYN attack script to be placed into the attacker containers.

This script would attempt to start a TCP 3 way handshake with a target IP address, and once it would be the attackers turn to do a ACK message to start sending data, the attacker would simply close that connection, forcing the victim to have to wait before closing their connection and serving others. Using this script would allow us to send less attack messages to the target which in turn would avoid the possibility of the DDoS attack messages from failing to reach the victim website. However this script failed when testing. Ben tested this scipt on his personal machine and noticed that while the script works in the start as intended, reading through wireshark that SYN messages were sent to the website and SYN ACK messages returned sent back to Ben's computer. However SYN ACK messages stopped being received from the website after less than 10 iterations. We kept the script in the code folder for reference.

To see how we created the docker container victim-attacker setup, refer to the docx file submitted in our code folder titled "docker victim attacker container setup", here we labeled every step done to create each container, as well as where we placed attack scripts when testing.

VI. Theory and Research

It is imperative to achieve a cursory level of modern knowledge of both attacks and defenses against DDoS attacks. Understanding the intricacies of various attack vectors and defense mechanisms is important to safeguarding network infrastructures against malicious agents. This comes with understanding both attack and defensive measures. As such we will be breaking down each and describing vulnerabilities and methods therein.

A. Attacks

A distributed denial of service attack (DDoS) is an attack that targets vulnerable hardware on a network with the intent to disrupt services or communications throughout the network. This can be a small device such as a router, or a larger DNS service provider. The attacker(s) will typically construct a network similar to 3, in which distributed geographically will send instructions to handlers, and the handlers will send instructions to compromised agents/zombies that will then coordinate attacks on the target or victim.

These structures are typically called "botnets", and are geographically distributed to prevent a single targeted defense to make them ineffective. It is important to note that these botnets can also distribute malware such as Trojans or viruses to increase the size and remove itself from the compromised machine once it has completed its task. There are many forms of DDoS attacks and the taxonomy can be referenced in Figure 2. A few of these will be referenced in following sections.

There have been several well known DDoS attacks that come to mind immediately which deserve their own study but we will be summarizing their effect here:

- **EarthLink Spammer (2000):** One of the first well known botnet attacks according to the cybersecurity exchange.[5] First implemented with phishing attacks to cause vulnerabilities in machines.
- **Mafiaboy (2000):** Another one of the first well known DDoS attacks. This attack was orchestrated by a Canadian hacker who managed to take down common commercial websites like Amazon and eBay as well as several others. [10]
- **CloudFlare DDoS (2014):** An attack that occurred on the cloudflare servers utilizing NTP Amplification, peaked at 400 gbps. Spanning 4,529 NTP servers on 1,298 networks. [4]
- **Dyn Attack (2016):** Consisting of three consecutive DDoS attacks against the DNS provider *Dyn*. By using a Mirai botnet, the attackers were able to overwhelm the servers and disrupt services of most commonly used websites and services. [20]
- **Google (2017):** In September, a 6-month long DDoS attack resulting in an average of 2.5 Tbps being sent to the google servers. Spoofing 167 Mpps to exposed CLDAP, DNS and SNMP servers. The attack was 4 times larger than the previous high profile attack a year earlier and remains the largest recorded attack currently known. [3]
- **Github Attack (2018):** About a thousand automated machines (ASNs) across many enpoints attempted a DDoS attack on Github by using a memcashed-based amplification attack. Resulting in 1.35 Tbps with 126.9 Mpps. [11]

As you can see, these previous examples all consisted of varying methods and victims. But all could affect the services due to the clever nature of their implementation.

1) Attacker Methodology: According to Douligeris and Hoque the malicious DDoS attacker's aim is to disrupt a network or service, generally by following these rules of engagement [9][12]:

1) Gather information and scan networks to find vulnerable machines in which to launch an attack.
2) Compromises hosts, installs malware or other programs so that they can be controlled as handlers or agents. See Figure 3.
3) Coordinate or update the botnet, arranging the logistics of an attack utilizing the handlers and agents that are available. Logistics can be type of attack, how long it takes, which ports, etc. Communication between client and the botnet can be done through TCP, UDP or ICMP protocols. [9]
4) Commence an attack towards the target.
5) Attempt to clean up and remove any remaining evidence from the memory of the affected machines.

There are many, constantly evolving types of DDoS

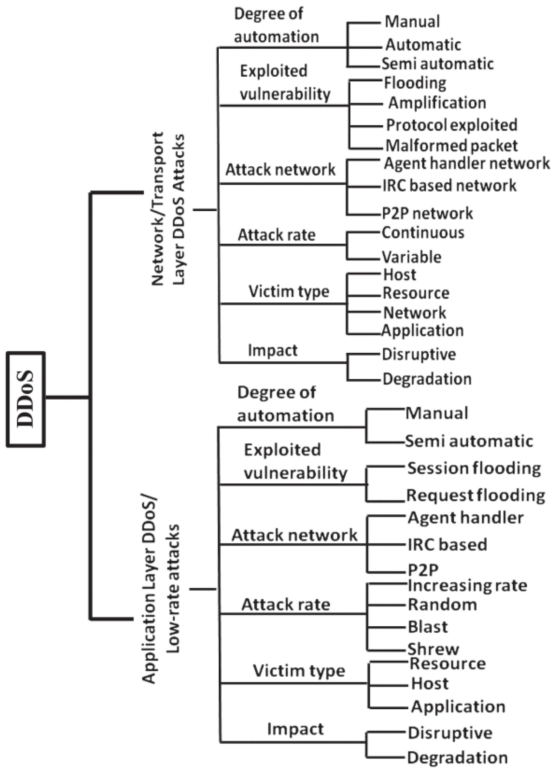

Fig. 2: An example of the taxonomy surrounding different DDoS Attacks and the reasons for their classification structure. [13]

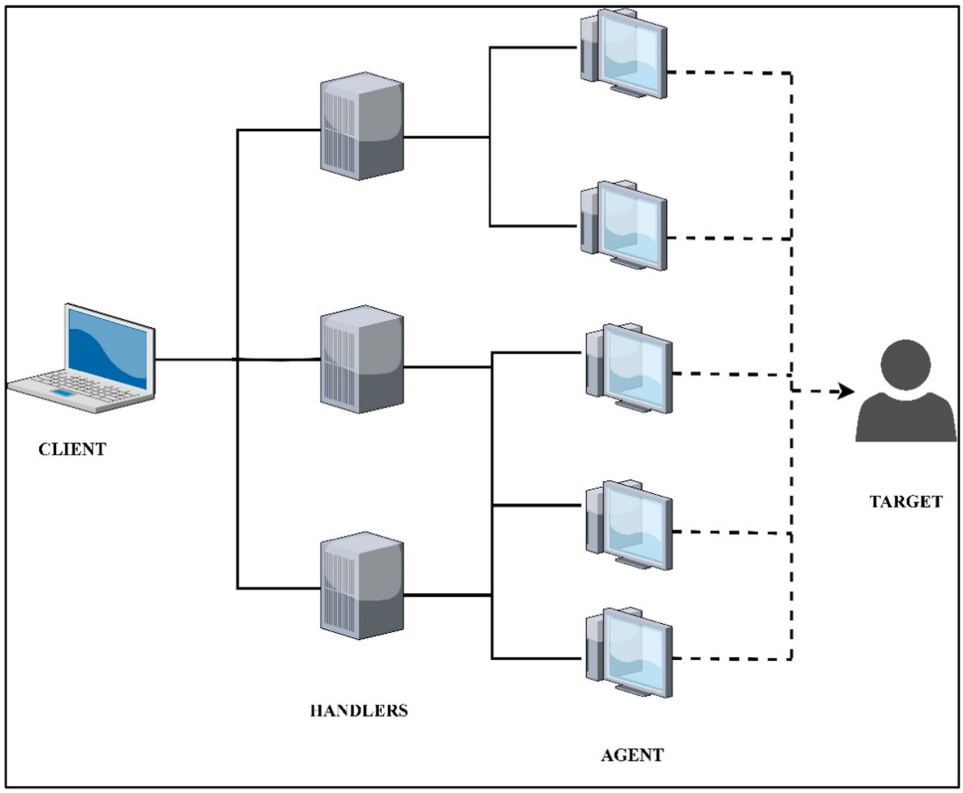

Fig. 3: An example of a standard DDoS Structure involving an attacker (client), handler machines, agents (zombies) and the target. [18]

attacks and infrastructures that can attack, such as (but not exhaustively): [9]:

- **Trinoo:** Is a trojan piece of malware that launches UDP flood attacks. [14]
- **TFN (Tribe Flood Network):** Produces ICMP flood attacks. [6]
- **TFN2K (Tribe Flood Network 2000):** A better TFN with additional features that encrypt and spoof it's trafficto avoid detection and retaliation. [19]
- **Stacheldraht:** AKA "Barbed Wire" use attacks to bring down website by flooding network traffic with a single client.[6]
- **mstream:** A tool that does TCP attacks as a point-to-point based on source code of "stream2.c". [8]
- **Shaft:** Modeled after Trinoo, consisted of UDP attacks. [7]
- **Trinity:** Launches several types of attacks including SYN, UDP,ACK and others. [17]
- **Knight:** Launches SYN and UDP flooding attacks, reported in July 2001. [9]

With the ever evolving cat and mouse game, we need to shift our focus on how to adapt quickly to the changing environment of network attacks, and learn how to defend against such intrusions when they occur.

B. Defenses

With the landscape of of threats always evolving, this necessitates deep understanding of current attack methodologies and also the development of defensive mechanisms that can anticipate and quickly eliminate the threat, if any, occur.

While typically the best network defense is to have no intrusion. This is impossible to do if the network needs to communicate. Here are some well known options for defensive measures.

1) Botnet Defensive System (BDS): A main type of DDoS attack that is still used today is the Mirai Botnet. The botnet consists of, as explained in the attack section, a few layers of compromised machines. We have the attacker, who coordinates attacks through the handlers. The handlers control several agents or zombies, and they in turn direct their malicious attacks towards the victim machine.

These DDoS attacks are varied widely in scope and effectiveness however their general intent is to flood the victim machine with requests. All machines that handle network traffic can only handle so many requests at once. If we use an analogy of water drainage systems, a small gutter can't handle a large flood. Thus this is similar to request flooding as well, where the system will be unable to handle too many requests at once. In order to combat this threat there are several defensive measures that can be put in place.

A Botnet Defensive System (BDS) is a defensive method developed by Shingo Yamaguchi specifically in order to counter the pure ideology of mass bot-net attack. It is able to defend any IoT systems against any type of attack coming from the malicious bots. BDS applies the concept of fighting poison by counteracting it with an equivalent poison, and in this case, the BDS system will generate and deploy army of white-hat bot nets to detect and neutralize the malicious bots. [22]

2) BDS Infection Rate Analysis: In order to understand the effectiveness of a BDS system, we can analyze the infection/spread rate of the white bot in comparison to the malicious bot once. According to Yamaguchi's work, it is possible to grasp the impact of the system using the initial starting state and the general infection rate before and after self-termination around the 10,000 steps. The formula to generate the infection rate is described below.[23]

Description of Variables:

- $R_{mal}(t)$: infection rate of malicious bots.
- $\#_{obsmal}$: No of observable malicious bots.
- $\#_{unobsmal}$: No of un-observable malicious bots.
- $\#_{inobsmal}$: No of in-observable malicious bots.
- $R_{white}(t)$: infection rate of white-hat bot.
- $\#_{obswh}(t)$: No of observable white bots.
- $\#_{unobswh}(t)$: No of un-observable white bots.
- $\#_{inobswhy}(t)$: No of in-observable white bots.
- $\#_{node}$: total No of nodes.

$$R_{mal}(t) = \frac{\#_{obsmal}(t) + \#_{inobsmal}(t) + \#_{unobsmal}(t)}{\#_{node}}$$

$$R_{white}(t) = \frac{\#_{obswh}(t) + \#_{inobswh}(t) + \#_{unobswh}(t)}{\#_{node}}$$

3) BDS Simulation: The simulation will perform three times each with different white-hat bot life span, 3 4 and 5. A thousand runs for each life span simulation and record the average out of the 1000. We will perform this twice; BDS regard to the system network structure VS BDS in a general cases. [23]

A BDS's operation consists of the following four, different stages:

1) **The Monitoring Process:** It is the starting process. The system will initiate a global scanning process throughout different devices to detect the presence of any malicious bots. Due to the scale of some system, it is unlikely to be able to observe all possible nodes, which created three different section, observable section, un-observable section, and in-observable section, section that will be later detected from the deployed white hat bots. (Figure 4)
2) **The Strategy Planning and Bot Deployment**: With the knowledge of the malicious bots' existence, the BDS will study the type of attack those bots inherit and the pattern in its location. Eventually, the system will come up with the counter strategy and launches the perfect quantity of white hat bot into the network for the infected systems. (Figure 5)
3) **Command and Control**: After deployment, the initial amount of the white hat bots will gradually spreading throughout the systems, "infecting" the maliciously "infected" nodes and terminate the malicious bots. This process helps fixing the setback in the monitoring process. The spread of the white hat allows the BDS to detect most of the infected nodes in the un-observable section and terminate it. All the generated white hat bots will form an inter-link with each other. (Figure 6)
4) **Self-destruct**: After a certain set amount of time, all the bots will self-terminate itself in order to save machine resources. (Figure 7)

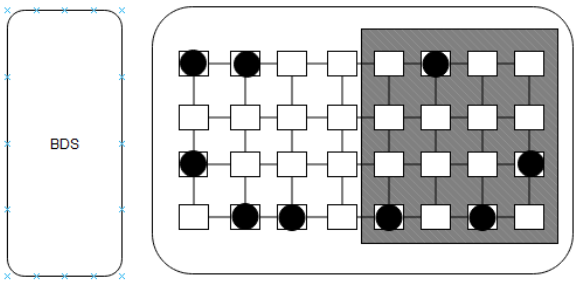

Fig. 4: Observable nodes vs Un-observable (Shaded region)

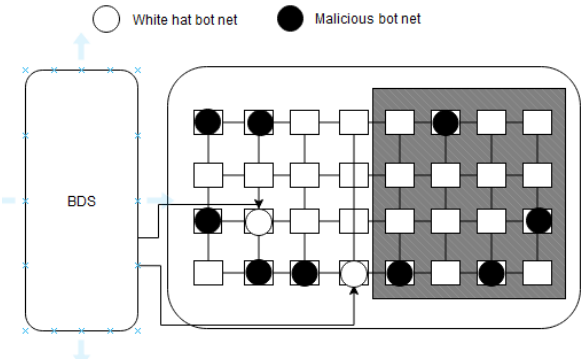

Fig. 5: White-hat bot deployment

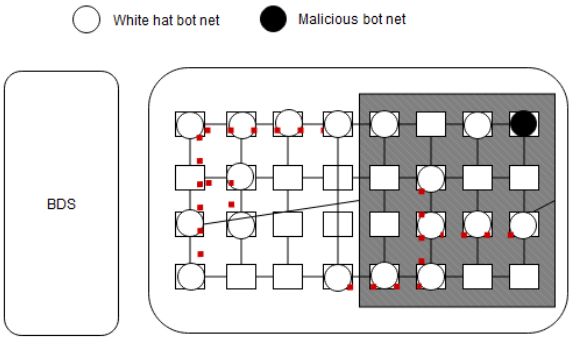

Fig. 6: White-hat bots autonomously expanding itself to multiple unit to terminate the malicious bot while forming an inter-link

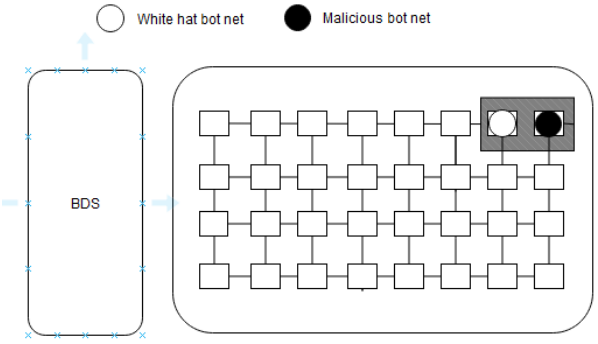

Fig. 7: Self-termination

4) Entropy-based Detection: One method we expect to be used to detect DDoS attacks is to use entropy-based approaches to calculate instantaneous summary statistics about the state of the network and compare those to predefined thresholds. In the context of cybersecurity, entropy is relatively inexpensive computationally and a measure of how random certain metrics on a network are.

Specifically, probability distributions are created and assess the probability that an attack is happening given the locations of IP addresses connecting to the site and the port numbers being used.[2] Entropy-based techniques have been used to determine the randomness of data flowing into servers to identity anomalies and potentially malicious attacks.[21]

5) Machine Learning-based Detection: A second approach involves using machine learning-based methods. These techniques use Bayesian networks and Self-Organizing Maps (SOMs). Bayesian networks can be trained on historical network traffic data to learn the normal behavior of the network. We expect the Bayesian network to evaluate new network traffic data and compute the probability that the observed behavior is normal or anomalous based on the learned model.

SOMs are a type of artificial neural network that uses unsupervised learning to map high-dimensional data onto a lower-dimensional grid. SOMs create a map of the structure of the network (i.e. the servers and other devices). The SOM knows what the distribution of normal behavior on the network looks like and can use this to identify irregular distributions and thus irregular behavior on the network[2].

Previous work has shown that, although Bayesian inference may be useful to detect intrusions, the accuracy of the Bayesian model at determining whether or not an intrusion is occurring is inversely proportional to how aggressively the model attempts to detect intrusions.[1]

6) Traffic Pattern Analysis: A third method, traffic pattern analysis, makes use of the fact that machines that are part of a DDoS attack typically act identically and in tandem in contrast to non-malicious machines on the network. Because bots on a botnet frequently work dependently upon a single controller, it can be possible to identify traffic patterns executed by many machines at the same time, thus enabling identification and targeting of these botnets.[2]

OpenFlow is a protocol developed for regulating incoming traffic. OpenFlow controllers are capable of detecting botnet actions and dropping, forwarding, or modifying incoming packets. These controllers can take the form of routers, switches, or firewalls. Malware programs are capable of infecting mobile devices by pretending to come from a legitimate source, however there are still other ways of targeting such traffic.

One research group found that maintaining a blacklist of malicious IP addresses can be used to quickly detect traffic from IP addresses. They found that implementing this strategy led to a decrease in the amount of malware targeting mobile devices.[15]

In the context of our research, understanding these defensive mechanisms provides valuable insights into potential countermeasures deployed by the target website, especially considering its operation through a Cloudflare proxy. By anticipating the defensive strategies employed, we can develop more informed attack algorithms and refine our approach to minimize detection and maximize impact on the target website.

VII. Results

Using the docker environment setup for attackers and defenders we were able to obtain information regarding simplistic DDoS attacks to a victim through a series of concurrent virtual systems. We will explain in the following section as to how we were able to find a valid destination IP address for our DDoS attack. We will also show the data we collected from the local victim-attacker container pairs, as well as the attacker information on the agora website created by Althea and Franklin.

A. Ethically Locating and Attacking AGORA Website by Franklin and Althea

For a successful DDoS attack to occur we need a valid IP address to use as a destination to send all of our DDoS related messages. Our group however came across the issue that the agora website has a IP address that is hidden by a Cloudflare proxy server.

According to Cloudflare Docs, ”If the domain’s status is active and the queried DNS record is set to proxied, then Cloudflare responds with an anycast IP address, instead of the value defined in your DNS table. This effectively re-routes the HTTP/HTTPS requests to the Cloudflare network, instead of directly reaching the targeted the origin server.” This means that without the hidden IP address we have no direct way of sending DDoS messages to the website.

One approach our group took was to try and use a trace route to helps us attempt to see what route packets may try to go. We believe that while we don’t have the destination IP address for the website, we believe that looking at the route that our packets went may give us insight as to a potential subroute which could lead us to the actual websites IP address.

However this attempt was shut down rather quick as a traceroute only gives us IP addresses. We need a better method which can help us correlate which IP addresses may belong in the same subdomain as the website

A second approach was to try and see if our group could trick the website into accidentally giving us its hidden IP address. This approach seemed plausible as it would allow us to avoid dealing with the Cloudflare proxy directly. Looking at the agora website we noticed that it has a email tool setup to help users create a account with the website.

Our group created a account and viewed the message details of the automatic email sent by the website. We were hoping that the website was using its hidden IP address as a source for the email, however when checking the email it appears that the website does not use its own IP address as a source for sending out account emails.

Instead, the website makes use of Mailgun, which is a separate email delivery service. This means that any IP address found in the email will not help us in finding a IP address that can be used to stage a DDoS attack on the agora website.

Our group also tried sending a email to postmaster@agora-app.com, a email that appeared in the description of the account setup email, however this also lead to no conclusions worth describing.

We also attempted to use CloudFail, which is a tool that brute force searches subdomains, misconfigured DNS scans, and uses a Crimeflare database to detect IP addresses covered by a Cloudflare proxy. However when We scanned the website, it detected over 12000 subdomains, however this method concluded to no insight of any hidden IP addresses, as after running Cloud Fail using the command ”python3 cloudfail.py –target agora-app.com”, nothing was returned

Another attempt at finding the hidden IP Address of the agora website was to try to use SecurityTrails, which is a ”Data Security company... with passive DNS servers, historical DNS records, reverse DNS, IP blocks...” SecurityTrails will allow our group to use past DNS records to try and see if any web hosts who used the currents site had a IP address not blocked by CloudFlare.

If this were true than it is possible that we could use that historical IP address as a potential destination for our DDoS attack. Looking in Security Trails, using ”agora-app.com”, SecurityTrails detected 8 different subdomains. Our group manually looked into each one until we checked a subdomain named ”dev2.agora-app.com”.

Looking at this and checking its DNS A records we saw three IP addresses, two of which were the same Cloudflare IP addresses seen for the original website, but now a new IP address was showing for a new organization marked as ”DigitalOcean,LLC”, with the value 164.92.78.11, checking a reverse IP lookup on this address we found three domain names.

Manually checking all three we found a domain name ”dev1.agora.narfnilk.com” which brought our group to what appears to be a development form of the agora website, with debugging posts and profiles. Pinging dev1.agora.narfnilk.com gives us the unprotected IP address 164.92.78.11. We also believe CloudFlare has no knowledge of this IP address as we tried running CloudFail on dev1.agora.narfnilk.com, and it did not detect that domain under CloudFlare.

B. Data

The following figures represent a multitude of different measurements resulting from our tests of our DDoS script on 1) the docker containers and 2) Franklin and Althea’s website. For figures 8, 9, 10, C = concurrency

or number of requests to work on at a time, n = number of total requests.

The effect of concurrency on the containers meant time to completion of HTTP requests was highly dependent on the total number of requests. For 10,000 concurrent HTTP requests, the difference between the time to completion of 50% and 100% for the of requests was 115 milliseconds, equivalent to 25.0% of the overall time required (Fig 8, Table II). In contrast, for 100,000 HTTP requests, that same difference was 3960 milliseconds, equivalent to 81.4% of the overall time required (Fig 9, Table 3).

The difference in time elapsed to process 50% and 60% of the requests is less than the difference in time elapsed between 99% and 100% for 100,000 requests. This suggests that, with a high number of requests and a high amount of concurrency, the bottleneck is due to the large amount of time needed for a small proportion of the total requests. These results show that concurrency is a major factor in the success of a DDoS attack.

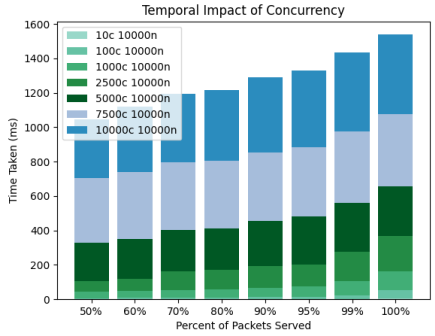

Fig. 8: Increasing concurrency slightly increases the time used for 10,000 packets. The height of each color represents the magnitude of time it took in ms.

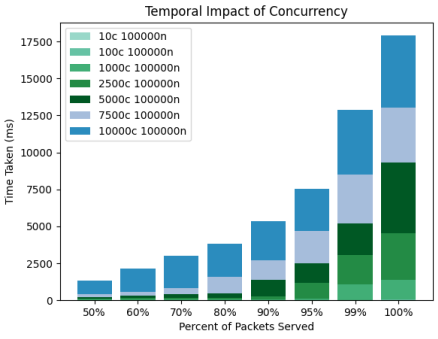

Fig. 9: Increasing concurrency results in more time used for 100,000 packets in an exponential fashion. The height of each color represents the magnitude of time that that percentage of packets took in ms.

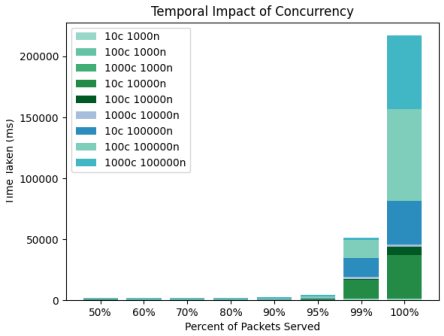

Fig. 10: Comparison in a real world Cloudflare proxy website against Figure 8 and Figure 9.

Figure 11, 12 and 13 show the total processing time and just the connection time, respectively, on a local machine using docker containers. Figure 11 and 12 validate the effectiveness of the DDoS script. As the number of concurrent requests increases, so does the amount of time needed. As shown in 13, the number of concurrent requests increases exponentially, the connection time also increases exponentially.

As was the case with the website, there exists for the docker containers a threshold which, upon being reached, substantially increases both the mean connection time and the variance in the connection time. In this case, that point is reached when the 10,000 requests are all sent concurrently. At that point, it is suspected that it takes a longer time for all packets to be processed by the server.

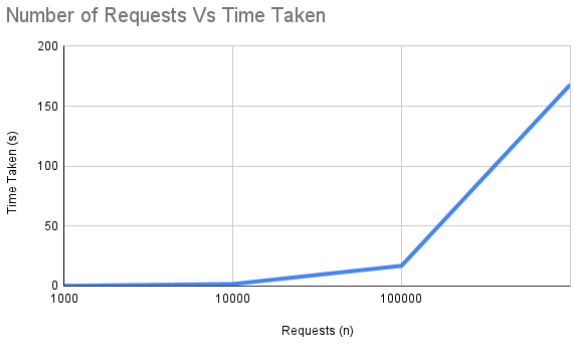

Fig. 11: Exponential increases in the number of concurrent requests result in exponential increases in the time taken. A DDoS attack was run on docker containers using 1,000, 10,000, 100,000 and 1,000,000 concurrent requests.

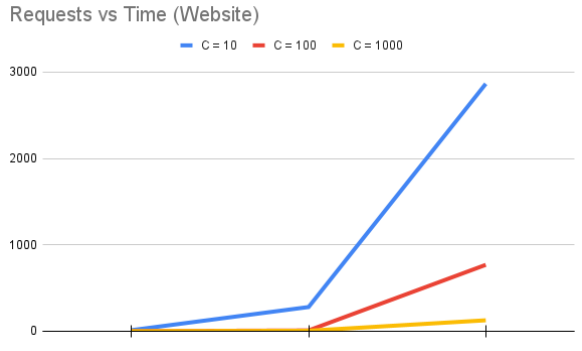

Fig. 12: Comparison in a real life Cloudflare proxy server behavior vs Figure 11 simulated behavior. Notice how similar they behave in that the time taken seems to stay the same until a request threshold is achieved.

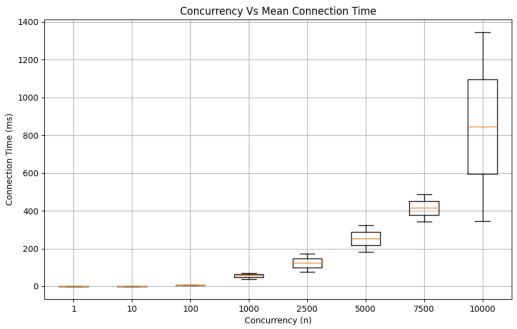

Fig. 13: Exponential increases in the number of concurrent requests result in exponential increases in the mean connection time. Boxes represent the interquartile range and the mean time, and whiskers represent the 1.5Q range. Run on the docker containers.

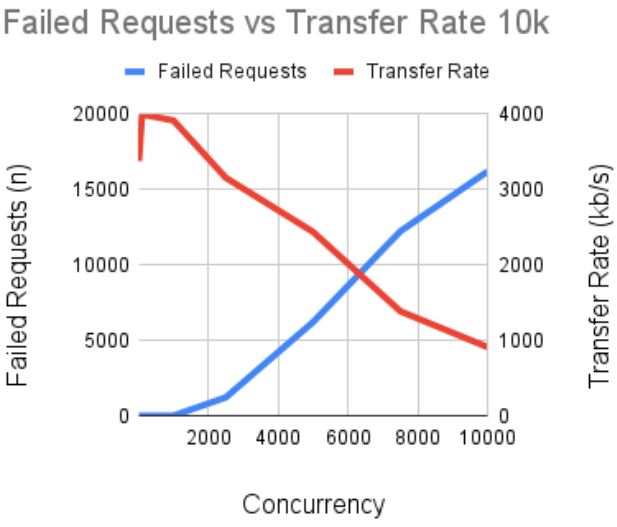

Fig. 14: As concurrency increases, the number of failed requests increases and the transfer rate drops. Shown are the number of failed requests and transfer rate in kb/s for 10,000 requests. Separate trials used 10, 100, 1,000, 2,500, 5,000, 7,500, or 10,000 concurrent attackers.

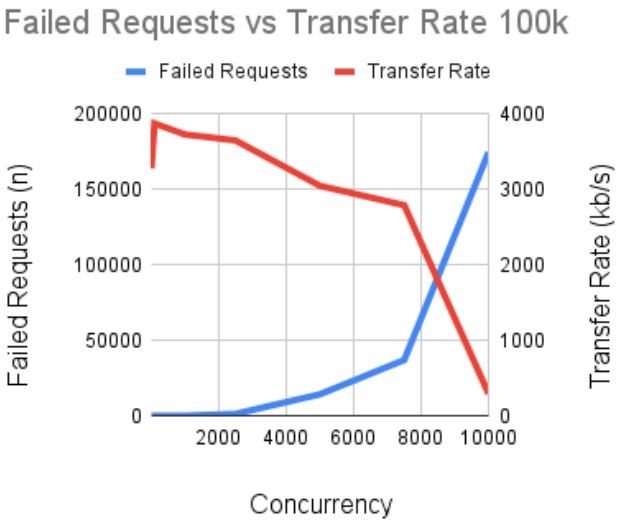

Fig. 15: As concurrency increases, the number of failed requests increases and the transfer rate drops. Shown are the number of failed requests and transfer rate in kb/s for 100,000 requests. Separate trials used 10, 100, 1,000, 2,500, 5,000, 7,500, or 10,000 concurrent attackers.

For DDoS attacks on the website server, it was observed that as the amount of concurrency increased, the transfer rate decreased and the number of failed requests increased, both linearly (Fig 14 and Fig 15). Notably, when using 10,000 concurrent attackers and 100,000 total requests, the transfer rate precipitously declined from 2790.81 under 75,000 total requests to 290.85 with 100,000 requests. If the number of requests further increased, it may be expected that the transfer rate may approach 0 kb/s.

At the same time as the transfer rate declined, the failed request rate experienced a spike. For the 100,000 total requests, 36,923 failed for 7,500 concurrent requests while 174,686 failed for 10,000 concurrent requests, an increase by 473%. As the number of failed requests appears to be increasing exponentially as the concurrency increases for 100,000 total requests, it may be expected that the rate of failure may continue to increase exponentially.

Taken together, these results suggest that the server's ability to handle requests declines as the concurrency increases, but rapidly declines if the total number of requests and concurrency exceed the server capacity threshold.

C. BDS Data

The result given from the simulation on the BDS system for the general cases and BDS system for known system network structure both look promising. It is more efficient when the white-hat bots lifespan is longer considering situations where the shorter life-span white-hat bots will self-terminate before the infection process completion.

With known system network structure, there is an increase of 400% and 115% in infection rate considering the change from life span length of 3 to the life span length of 4 and 5. There is also zero existence from malicious bot at the end of the simulation for life span of 5.

A similar trend applies for the general cases. Despite the different in the starting white-hat bots quantity, the BDS system still proposes a promising results as there is 0% of the malicious bots.

VIII. Conclusion

In conclusion, the work done here highlights the importance of education among those looking to defend against DDoS attacks. Through our research we were able to gain critical knowledge where the field of network security is involved. While mostly skin deep, the importance of a good background is paramount and our research generally supports the research done by professionals. In that there are a wide range of tools for both attack and defenses.

Attacks can lead to critical failure of victim resources. In our experimentation in the Docker setup, we had an unprotected victim producing valuable data for us, however it is important to note that when applied in the real world protections abound and while we got very similar results it wasn't perfect replication of our isolated testing. It's this difference between results that leads us to believe that there are more factors at play outside of the simple DDoS attack and measurements.

We had an interesting breakthrough in pinpointing the webpage of our peers, but there are factors that complicate it. Given the results we received, it's obvious that in some way the benchmark tool we used showed results slowed down, but what's less obvious is the route or potential hiccups that request took along the way. For example, there is the potential that the bottleneck was on the attacking machine instead.

In the sense of noticeable impact, the webpage was indistinguishably slower to the naked eye, but the report also considers the intrinsic protections offered by the host, of which a malicious party would have little knowledge on. Regardless, the findings of this paper present themselves as a success for better understanding the capability and resources required to wage or defend against a DDoS attack.

The study of DDoS protections and attacks are is widely ranging and even more insightful. Thankfully though, with the security practices of most of the modern internet, there are no more 'easy' targets. The findings of this report actually aid the idea that modern attacks are something of a marvel on their own. To launch a DDoS attack for 423 hours [16] for instance, is extremely impressive, and extremely scary.

Luckily, there are ample opportunities in our fight against malicious entities as discussed above, and we're confident that this report demystifies DDoS attacks for introductory individuals and makes plain what they are, and are not, capable of. Certainly from our preliminary tests, we're not currently in the same realm as the big players.

IX. Future Work

Our plans on future work is varied but if given the time there are several things we would like to do as a team:

1) Expand tests with available DDoS tools and package delivery devices.
2) Observe behavior across protected and unprotected networks on a dummy website.
3) Machine learning methods to detect malicious intrusion into compromised machines to dynamically shut off access in real time for damage mitigation.
4) More varied and in depth benchmarking, particularly when attacking a real server such as our peers website.
5) Implementation of more DDoS attacks. For instance, a slow feed DDoS attack may be interesting to view the long term behavior of.

X. Team Breakdown

- J. Middleman (Grad)
  - Research of DDoS defenses and detection, completing the non-botnet section of the defenses section of the report.
  - Scanning of Franklin & Althea's website to determing if the website server had any identifiable vulnerabilities. No vulnerabilities were found using NMap.
  - Subsequent validation assessment of our DDoS code was run using ApacheJ. This validation showed that a similar number of failured requests and a decrease in transfer rate occurred with use of ApacheJ.
  - Testing the docker code and helping to write the README file for the code repository.
  - Helping to write the results section of the report, specifically writing the non-botnet section of the text in this section.
  - Writing the text and finding images in the defense and results section of the presentation
- B. Ogden (Grad)
  - Implementation of DDoS docker environment setup, as well as data gathering from agora website made by Althea and Franklin
  - Worked on SYN attack python script
  - Attempted DDoS attack on website
    - Checked email service on AGORA website in attempt to find website IP address hidden behind Cloudflare proxy
    - Used CloudFail and SecurityTrails to find DNS records/ historical data to in attempt to find hidden IP address of website
    - Attempted trace route to find hidden IP address of website
  - Worked on Methodology and Results Section of Paper
  - Organized adding all code to git repo and Code folder for submission
- M. Rackley (Grad)
  - DDoS data generation
  - Taxonomy of DDoS attacks
  - Historical DDoS events
  - Attacker Methodology
  - Data visualization
  - Latex Report Sections:
    - Abstract
    - Introduction
    - Hypothesis
    - Methodology
    - Theory
    - Results
    - Conclusion
    - Future Work
    - References/Bib.tex
- L. Su (Undergrad)
  - Research of DDoS defenses and detection methods
  - Botnet implementation process research
  - Reported on the BDS data.
  - Yamaguchi's BDS analysis regarding it effectiveness through infection rate.
  - Yamguchi's BDS methodologies
- C. Tye (Grad)
  - Collaborated
    - DDoS Docker Environment Setup
    - Data Visualization
    - Presentation Formatting and Motivation
  - Led
    - DDoS Data Generation - Docker
    - Data Conceptualization
    - Introduction
    - Motivations
    - Report Formatting
    - Report Validation and Consistency
    - Conclusions
    - Future Works

REFERENCES

[1] S. Axelsson. "The base-rate fallacy and its implications for the difficulty of intrusion detection". In: *Proceedings of the 6th ACM Conference on Computer and Communications Security (CCS '99)*. 1999.

[2] N. Z. Bawany, J. A. Shamsi, and K. Salah. "DDoS Attack Detection and Mitigation Using SDN: Methods, Practices, and Solutions". In: *Arabian Journal for Science and Engineering* 42.2 (2017), pp. 425–441. DOI: 10.1007/s13369-017-2414-5.

[3] Google Cloud. *Identifying and protecting against the largest DDoS attacks*. Accessed on April 22, 2024. n.d.

[4] Cloudflare. *Technical details behind a 400Gbps NTP amplification DDoS attack*. Accessed on April 22, 2024. n.d. URL: https://blog.cloudflare.com/technical-details-behind-a-400gbps-ntp-amplification-ddos-attack/.

[5] EC-Council. *The Biggest Botnet Attacks to Date*. Accessed on April 22, 2024. n.d.

[6] Paul J Criscuolo. "Distributed denial of service: Trin00, tribe flood network, tribe flood network 2000, and stacheldraht ciac-2319". In: *Department of Energy Computer Incident Advisory Capability (CIAC), UCRL-ID-136939, Rev* 1 (2000).

[7] Sven Dietrich and Neil Long. "Analyzing distributed denial of service tools: The shaft case". In: *14th Systems Administration Conference (LISA 2000)*. 2000.

[8] David Dittrich et al. "The mstream distributed denial of service attack tool". In: *University of Washington, http://staff. washington. edu/dittrich/misc/mstream. analysis. txtS* (2000).

[9] Christos Douligeris and Aikaterini Mitrokotsa. "DDoS attacks and defense mechanisms: classification and state-of-the-art". In: *Computer Networks* 44.5 (2004), pp. 643–666. DOI: https://doi.org/10.1016/j.comnet.2003.10.003. URL: https://www.sciencedirect.com/science/article/pii/S1389128603004250.

[10] Gary Genosko. "The Case of 'Mafiaboy' and the Rhetorical Limits of Hacktivism". In: *Fibreculture Journal* (Jan. 2006).

[11] GitHub. *DDoS Incident Report*. Accessed on April 22, 2024. Mar. 2018.

[12] N. Hoque, D. K. Bhattacharyya, and J. K. Kalita. "Botnet in DDoS Attacks: Trends and Challenges". In: *IEEE Communications Surveys & Tutorials* 17.4 (Fourthquarter 2015), pp. 2242–2270. DOI: 10.1109/COMST.2015.2457491.

[13] Nazrul Hoque, Dhruba K. Bhattacharyya, and Jugal K. Kalita. "Botnet in DDoS Attacks: Trends and Challenges". In: *IEEE Communications Surveys & Tutorials* 17.4 (2015), pp. 2242–2270.

[14] Abhishek Jain et al. "Defending distributed denial of service (Ddos) attacks: Classification and art". In: *Ilkogretim Online* 20.2 (2021), pp. 2498–2509.

[15] R. Jin and B. Wang. "Malware Detection for Mobile Devices Using Software-Defined Networking". In: *2013 Second GENI Research and Educational Experiment Workshop*. 2013.

[16] Kaspersky. *DDoS Attacks in Q2 2022*. 2022. URL: https://securelist.com/ddos-attacks-in-q2-2022/107025/ (visited on 04/28/2024).

[17] Gary C Kessler. "Defenses against distributed denial of service attacks". In: *SANS Institute* 2002 (2000).

[18] Pooja Kumari and Ankit Kumar Jain. "A comprehensive study of DDoS attacks over IoT network and their countermeasures". In: *Computers & Security* 127 (2023), p. 103096. DOI: https://doi.org/10.1016/j.cose.2023.103096. URL: https://www.sciencedirect.com/science/article/pii/S0167404823000068.

[19] Asma Mumtaz. "Development of State-of-Art Distributed Denial of Service (DDoS) Attack Generation and Mitigation Tool". In: (2008).

[20] TechCrunch. *Many sites, including Twitter and Spotify, suffering outage*. Accessed on April 22, 2024. Oct. 2016.

[21] R. Wang, Z. Jia, and L. Ju. "An entropy-based distributed DDoS detection mechanism in software-defined networking". In: *2015 IEEE Trustcom/BigDataSE/ISPA*. 2015, pp. 310–317.

[22] Shingo Yamaguchi. "A basic command and control strategy in botnet defense system". In: *2021 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE. 2021, pp. 1–5.

[23] Shingo Yamaguchi. "Botnet Defense System: Observability, Controllability, and Basic Command and Control Strategy". In: *Sensors* 22.23 (2022), p. 9423.

TABLE I: Initial test run

C	R	Time	RPS	KB/s	Mean	Time (ms) To Serve Percent							
						50	60	70	80	90	95	99	100
1	1K	0.199	5031	1390.53	0	0	0	0	0	0	0	1	1
1	10K	1.679	5956.16	1646.09	0	0	0	0	0	0	0	0	1
1	100K	16.904	5915.86	1634.95	0	0	0	0	0	0	0	0	55
1	100K	168.078	5949.63	1644.28	0	0	0	0	0	0	0	0	55
10	10K	0.69	14488.91	4004.26	1	1	1	1	1	1	1	2	5
100	100K	5.813	17201.41	4753.91	6	5	6	7	8	9	10	20	42
1000	1M	58.498	17094.47	4724.35	58	39	47	51	54	60	66	1053	2257

TABLE II: Run with 10,000 requests.

C	R	Time	RPS	KB/s	Mean	Time (ms) To Serve Percent							
						50	60	70	80	90	95	99	100
10	10K	0.818	12220.26	3377.28	1	1	1	1	1	1	1	2	5
100	10K	0.692	14454.44	3994.73	7	6	7	8	9	11	13	20	50
1000	10K	0.706	14164.55	3914.62	40	37	42	46	48	55	60	82	107
2500	10K	0.823	12154.08	3154.76	76	61	68	109	113	124	128	173	207
5000	10K	0.782	12756.64	2436.91	182	225	234	238	240	266	280	285	289
7500	10K	0.776	12887.64	1384.8	342	376	387	392	394	398	402	413	419
10000	10K	0.579	17285.82	909.59	346	339	379	401	410	435	447	457	461

TABLE III: Run with 100,000 requests.

C	R	Time	RPS	KB/s	Mean	Time (ms) To Serve Percent							
						50	60	70	80	90	95	99	100
10	100K	8.424	11870.9	3280.73	1	1	1	1	1	1	1	3	10
100	100K	7.133	14020.2	3874.72	7	7	8	9	9	10	11	18	60
1000	100K	7.412	13491.83	3728.7	70	45	51	55	59	70	80	1076	1303
2500	100K	7.527	13285.34	3649.56	173	72	89	101	109	160	1111	1957	3161
5000	100K	8.402	11902.55	3048.29	323	88	159	257	294	1144	1309	2141	4788
7500	100K	8.045	12430.37	2790.81	488	195	266	386	1105	1327	2161	3312	3712
10000	100K	13.836	7227.48	290.85	1345	903	1589	2224	2270	2662	2869	4345	4863

TABLE IV: Data on a DDoS Attack on Website

R	C	Time	RPS	KB/s	Mean	Time (ms) To Serve Percent								
						50	60	70	80	90	95	98	99	100
1000	10	12.293	81.34	68.08	114	72	72	773	74	75	306	1072	1074	1078
1000	100	0.874	1144.31	957.69	72	71	71	72	72	74	82	86	89	92
1000	1000	0.478	2091.72	1750.59	261	266	291	309	323	337	342	345	346	348
10000	10	280	35.67	29.85	276	72	72	73	73	74	307	1072	15072	35549
10000	100	9.466	1056.37	884	92	93	95	96	97	100	102	105	108	1102
10000	1000	6	1679.3	1405	544	545	614	667	697	748	830	1134	1699	1885
100000	10	2863.481	34.53	28.89	289	72	72	72	73	74	75	1072	15066	35796
100000	100	768.363	130.15	108.53	661	72	72	73	74	78	1070	15067	15081	75073
100000	1000	126.628	789.72	660.68	640	471	600	633	657	745	850	1343	1830	60205

(a) Lifespan $\ell = 3$ Steps

Step t	Malicious Bots				White-Hat Bots			
	$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$
0^-	5.79	0.00	14.21	20.00%	0.00	0.00	0.00	0.00%
0^+	5.79	0.26	13.95	20.00%	10.00	0.00	0.00	10.00%
$10,000$	23.48	0.51	43.40	67.39%	7.55	6.17	3.08	16.80%
$10,000^+$	23.48	0.51	43.40	67.39%	0.00	0.00	3.08	3.08%

(b) Lifespan $\ell = 4$ Steps

Step t	Malicious Bots				White-Hat Bots			
	$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$
0^-	5.79	0.00	14.21	20.00%	0.00	0.00	0.00	0.00%
0^+	5.79	0.26	13.95	20.00%	10.00	0.00	0.00	10.00%
$10,000^-$	0.71	0.38	1.72	2.81%	29.81	33.06	10.41	73.28%
$10,000^+$	0.71	0.38	1.72	2.81%	0.00	0.00	10.41	10.41%

(c) Lifespan $\ell = 5$ Steps

Step t	Malicious Bots				White-Hat Bots			
	$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$
0^-	5.79	0.00	14.21	20.00%	0.00	0.00	0.00	0.00%
0^+	5.79	0.26	13.95	20.00%	10.00	0.00	0.00	10.00%
$10,000^-$	0.00	0.00	0.00	0.00%	32.72	43.62	8.18	84.52%
$10,000^+$	0.00	0.00	0.00	0.00%	0.00	0.00	8.18	8.18%

Fig. 16: Simulation average data of the infection rate of malicious and white-hat bot at the initial state before/after white-hat deployment and at time $10,000^-$ / $10,000^+$ (before/after self-destruction) given the system network structure. [22]

(a) Lifespan $\ell = 3$ Steps

$\#^{obs}_{node}$	$\overline{\#_{wh}(0^+)}$	Malicious Bots					White-Hat Bots				
		Step $10,000^-$				Step $10,000^+$	Step $10,000^-$				Step $10,000^+$
		$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$	$\overline{R_{wh}}$
10	10	0.51	0.20	4.33	5.04%	5.04%	7.28	63.99	1.21	72.47%	1.21%
20	10	1.01	0.18	3.84	5.04%	5.04%	14.67	56.83	0.97	72.47%	0.97%
	20	1.49	0.34	5.66	7.49%	7.49%	14.22	55.78	0.83	70.83%	0.83%
30	10	1.52	0.17	3.35	5.04%	5.04%	21.96	49.75	0.77	72.47%	0.77%
	20	2.24	0.30	4.95	7.49%	7.49%	21.42	48.68	0.73	70.83%	0.73%
	30	2.38	0.11	5.45	7.94%	7.94%	20.49	47.33	0.56	68.38%	0.56%

(b) Lifespan $\ell = 4$ Steps

$\#^{obs}_{node}$	$\overline{\#_{wh}(0^+)}$	Malicious Bots					White-Hat Bots				
		Step $10,000^-$				Step $10,000^+$	Step $10,000^-$				Step $10,000^+$
		$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$	$\overline{R_{wh}}$
10	10	0.00	0.00	0.00	0.00%	0.00%	8.72	77.93	0.53	87.19%	0.53%
20	10	0.00	0.00	0.00	0.00%	0.00%	17.55	69.16	0.48	87.19%	0.48%
	20	0.00	0.00	0.00	0.00%	0.00%	17.38	69.08	0.36	86.83%	0.36%
30	10	0.00	0.00	0.00	0.00%	0.00%	26.30	60.52	0.37	87.19%	0.37%
	20	0.00	0.00	0.00	0.00%	0.00%	26.19	60.31	0.32	86.83%	0.32%
	30	0.00	0.00	0.00	0.00%	0.00%	26.18	60.64	0.29	87.11%	0.29%

(c) Lifespan $\ell = 5$ Steps

$\#^{obs}_{node}$	$\overline{\#_{wh}(0^+)}$	Malicious Bots					White-Hat Bots				
		Step $10,000^-$				Step $10,000^+$	Step $10,000^-$				Step $10,000^+$
		$\overline{\#^{obs}_{mal}}$	$\overline{\#^{in\text{-}obs}_{mal}}$	$\overline{\#^{unobs}_{mal}}$	$\overline{R_{mal}}$	$\overline{R_{mal}}$	$\overline{\#^{obs}_{wh}}$	$\overline{\#^{in\text{-}obs}_{wh}}$	$\overline{\#^{unobs}_{wh}}$	$\overline{R_{wh}}$	$\overline{R_{wh}}$
10	10	0.00	0.00	0.00	0.00%	0.00%	9.16	81.50	0.17	90.82%	0.17%
20	10	0.00	0.00	0.00	0.00%	0.00%	18.34	72.35	0.13	90.82%	0.13%
	20	0.00	0.00	0.00	0.00%	0.00%	18.15	72.62	0.14	90.91%	0.14%
30	10	0.00	0.00	0.00	0.00%	0.00%	27.47	63.23	0.12	90.82%	0.12%
	20	0.00	0.00	0.00	0.00%	0.00%	27.33	63.46	0.13	90.91%	0.13%
	30	0.00	0.00	0.00	0.00%	0.00%	27.25	63.40	0.10	90.74%	0.10%

Fig. 17: Simulation average data of the infection rate of malicious and white-hat bot at the initial state before/after white-hat deployment and at time $10,000^-$ / $10,000^+$ (before/after self-destruction) regardless of the system network structure. [22]

XI. Appendix

The following are larger images of the more important figures in the main text:

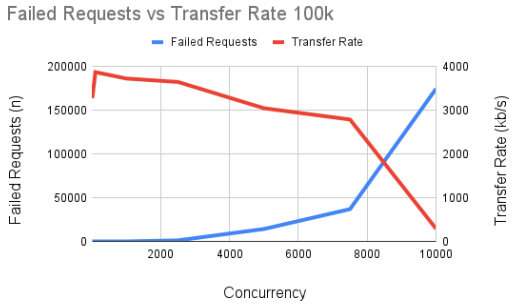

Fig. 18: A graph showing as concurrency increases for 100,000 requests, the number of failed requests increases and transfer rate drops as the target cannot handle the load.

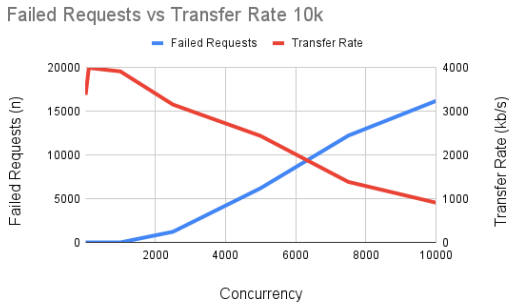

Fig. 19: A graph showing as concurrency increases for 10,000 requests, the number of failed requests increases and transfer rate drops as the target cannot handle the load.

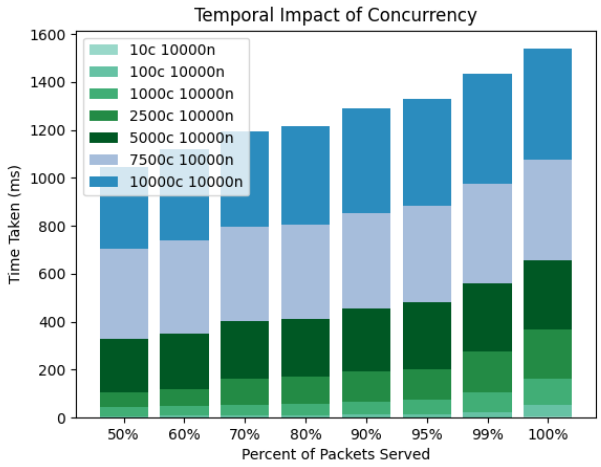

Fig. 20: While simple and can be inferred. Checking to ensure that the increased load of requests increases the time taken to be served.

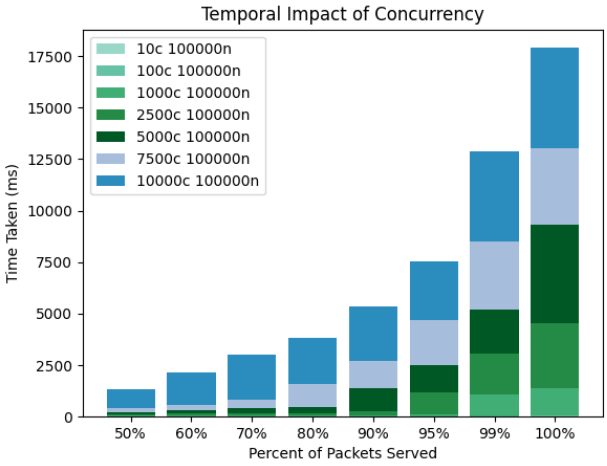

Fig. 21: While simple and can be inferred. Checking to ensure that the increased load of requests increases the time taken to be served.

A. Botnet Setup

We utilize an extremely well-known educational tools called BYOB. Not only this tools provides a working server, command and control within our own console, but it also provides a working Web-GUI with various impressive features and extremely friendly. The process in Linux:

1) Make sure to port forward ports 1337-1339, have python3 and pip installed.
2) Clone the repository, go to web-gui directory, run ”startup.sh”
3) Installed all the package require from ”requirements.txt”.
4) Run the ”run.py” script. This will set up the server for the web gui.
5) Login to the web site through the provided URL from ”run.py”
6) Register an account and login.
7) Go to payload section, generate a payload in an executable file that compatible with the system you want to embedded the botnet to. I.e: Linux amd64.
8) Once you successfully embedded the botnet into the victim sytem, you can perform multiple task anonymously such as Keylogger, Screenshot, Webcam, etc. There is fixed amount of feature. You can implement your own modules in python script such as the BDS system, and it will play as a defensive tools you can use to counter Ddos attacks.